

System Design Basics

Lesson 6: Load Balancing & Traffic Distribution

Load Balancing & Traffic Distribution – Study Notes

Key Concepts

1. **Load Balancer (LB)** – A device or software that distributes incoming client requests across multiple backend servers to improve performance, reliability, and scalability.
2. **Round Robin** – A simple algorithm that sends each new request to the next server in a circular list, regardless of current load.
3. **Least Connections** – An algorithm that forwards a request to the server with the fewest active connections (or lowest load metric).
4. **DNS Load Balancing** – Using the Domain Name System to return multiple IP addresses for a single hostname, letting clients pick one (often at random or based on resolver behavior).
5. **Health Checks** – Periodic probes that verify a server is up and healthy; unhealthy nodes are removed from the rotation automatically.

Summary

Load balancing is the cornerstone of modern, highly available web services. By spreading traffic across a pool of identical servers, a load balancer prevents any single machine from becoming a bottleneck, reduces response time, and provides resilience when a server fails.

Three introductory techniques are commonly taught:

- **Round Robin** is the easiest to understand and implement. It cycles through the server list in order, giving each server an equal share of requests. It works well when all servers have similar capacity and the workload per request is roughly constant.
- **Least Connections** adds a bit of intelligence: the balancer tracks how many active sessions each server is handling and sends new requests to the least busy node. This is useful when request processing time varies widely (e.g., some pages are CPU intensive, others are quick).
- **DNS Load Balancing** distributes traffic at the DNS resolution stage. The authoritative DNS server returns several A/AAAA records for the same hostname. Clients (or recursive resolvers) typically choose one address—often randomly or based on round robin ordering. DNS LB is cheap and works without dedicated hardware, but it lacks real time health checking and cannot react quickly to a server failure.

Together, these methods illustrate the trade offs between simplicity, responsiveness, and control that engineers must consider when designing a scalable web service.

Important Points

- **Stateless vs. Stateful**: Load balancers work best with stateless services. If sessions are stateful, you may need sticky sessions (session affinity) or a shared session store.
- **Health Checks**: Most LBs perform HTTP, TCP, or custom probes; a failed health check removes

the server from the pool until it recovers.

- **Round Robin Limitations**: Ignores server load, request size, and latency; can overload a slower node.
- **Least Connections Benefits**: Adapts to variable request lengths; may need a “connection weight” to account for servers with different capacities.
- **DNS LB Characteristics**:
 - No built in health checking (unless using advanced DNS services).
 - TTL (time to live) controls how long clients cache the IP list; lower TTL = faster failover but higher DNS query volume.
 - Works at the *client* side, so the client may keep using a failed IP until its cache expires.
- **Layer 4 vs. Layer 7**:
 - L4 (Transport) LB forwards packets based on IP/port only – fast, but limited routing logic.
 - L7 (Application) LB can inspect HTTP headers, URLs, cookies, enabling content based routing and SSL termination.
- **Scalability**: Adding a new backend server usually requires only updating the LB pool (or DNS record) – no code changes to the application.

Examples

1. Round Robin with a Simple Web Service

Assume three web servers: `10.0.0.1`, `10.0.0.2`, `10.0.0.3`.

A Layer 7 reverse proxy (e.g., Nginx) is configured:

```
```nginx
http {
 upstream webfarm {
 server 10.0.0.1;
 server 10.0.0.2;
 server 10.0.0.3;
 }
 server {
 listen 80;
 location / {
 proxy_pass http://webfarm;
 }
 }
}
```
```

Nginx's default `least\_conn` is **off**, so it uses round robin.

Request #1 !' 10.0.0.1, Request #2 !' 10.0.0.2, Request #3 !' 10.0.0.3, then repeats.

If `10.0.0.2` crashes, Nginx's health check marks it down, and the next request goes to 10.0.0.3, then 10.0.0.1, etc.

2. DNS Load Balancing for the Same Service

Authoritative DNS zone:

```
example.com. IN A 203.0.113.10 ;LB1
example.com. IN A 203.0.113.11 ;LB2
example.com. IN A 203.0.113.12 ;LB3
```

When a client resolves `example.com`, the DNS server returns the three A records in a rotating order.

If the client's resolver caches the answer for 60 /seconds (TTL /= /60), it will keep using the same IP for that minute even if one LB goes down. Reducing TTL to 5 /seconds improves failover but increases DNS traffic.

Quick Review

1. **What is the main difference between round robin and least connections algorithms?**
2. **Why might DNS load balancing be insufficient for mission critical services?**
3. **What role do health checks play in load balancing?**
4. **Name one advantage of a Layer 7 (application) load balancer over a Layer 4 (transport) balancer.**
5. **If a server handling long running requests crashes, which algorithm is more likely to keep overall response times low?**

Further Reading

- **Sticky Sessions / Session Affinity** – handling stateful web applications.
- **Weighted Round Robin & Weighted Least Connections** – balancing heterogeneous server capacities.
- **Consistent Hashing** – used by CDN edge nodes and distributed caches.
- **Cloud Native Load Balancers** – AWS ELB/ALB, Google Cloud Load Balancing, Azure Load Balancer.
- **Service Meshes (e.g., Istio, Linkerd)** – built in traffic routing, retries, circuit breaking.

End of notes.

